

DPP Assignment 3

Simon Vinding Brodersen(rzn875)

December 8, 2025

1 Tree Operations

1.1 Implement *depths*

Below my implementation of the depths can be seen.

```
def depths (steps: []step) : [](i64, i32) =  
  let (arr, values, keep) = unzip3 <| map (\s ->  
    match s  
      case #u -> (-1, -1, false)  
      case #d x -> (1, x, true)  
  ) steps  
  let inc_scan_arr = scan (+) 0 arr  
  let exc_scan_arr = map2 (\a b -> a - b) inc_scan_arr arr  
  let (depth, value, _) = zip3 exc_scan_arr values keep |> filter (\(_, _, b) -> b) |>  
    unzip3  
  in zip depth value
```

Here we have to convert the steps into items of 1 for down and -1 for up operations. Then we can do an exclusive scan and at last remove all the elements which originally corresponded to up items as these do not add new elements to the graph.

If we let N denote the length of steps, then we have a map, a scan, another map and at last a filter, which all work on an array of the full length N . The filter and the scan operation is what is the limiting factor giving us work of $O(N)$ and span of $O(\lg N)$.